

Relatório 17 - Prática: Redes Neurais Convolucionais 2 (Deep Learning) (II)

João Pedro Gomes

1. Introdução

O card 17 ensina conceitos de redes convolucionais, explica como filtros em redes convolucionais funcionam, o que o padding pooling fazem, etc..., o card pede pra assistirmos 32 videos e fazermos uma parte prática do que foi ensinado e esse relatório.

2. Desenvolvimento

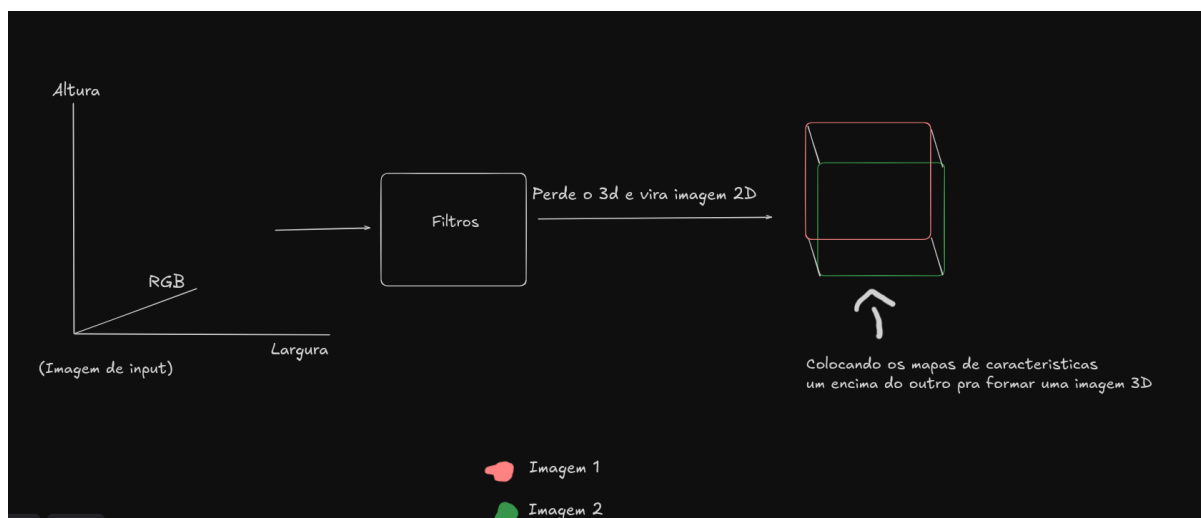
Primeiramente ele aborda novamente sobre redes convolucionais, ele começa explicando que convolução é o método onde se aplica um filtro em uma imagem pra extrair as características, como por exemplo pra verificar se tem um cachorro em uma imagem se usa redes convolucionais.

Depois ele aborda a forma de extrair as características, onde a imagem é jogada pra uma matriz e o filtro é uma matriz menor que multiplica áreas da imagem gerando uma matriz nova que guarda as características da imagem, e que pra isso tem técnicas de borrar a imagem ou detectar as bordas dela.

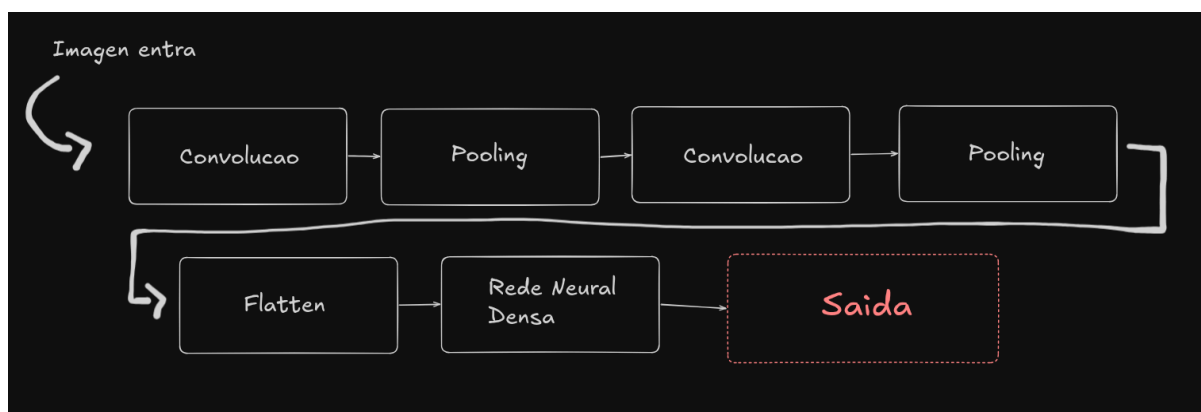
Porém depois de passar o filtro na matriz da imagem a matriz de saída é menor que a imagem de entrada, pra isso existe o padding que adiciona zeros nas bordas pra imagem não diminuir.

Diferente das redes neurais densas que analisa uma imagem como um monte de números e não identifica os padrões, uma rede convolucional analisa cada pedaço e vai aprendendo padrões e depois de treinada ela procura pelos mesmos padrões independente de qual local da imagem estejam.

Com imagens coloridas a matriz deixa de ser 2D e passa a ser 3D, pois tem o RGB como profundidade que são as cores da imagem porém, depois da passagem do filtro a matriz de características se transforma em 2D novamente, mesmo a matriz de input sendo 3D. Pra resolver isso é feito vários filtros pra pegar as características que depois são empilhadas junto pra formarem uma matriz 3D com profundidade.



A arquitetura de uma rede convolucional é formada por várias partes, sendo elas, a convolução, pooling, flatten, rede neural densa e a saída. A camada de convolução já foi explicada anteriormente, a camada de pooling onde ela diminui o tamanho da imagem pra economizar na hora das contas e ajuda a rede a ignorar a posição de onde algum padrão foi encontrado, se ele estiver buscando por um olho ele não liga se achou o olho no meio ou em algum canto, existe também o max pooling que analisa uma parte da matriz original da imagem e nos índices analisados ele pega o maior valor que mostra que a característica apareceu mais ali, depois das camadas de convolução vem o flatten que pega todos os dados das características e coloca em um vetor pra enviar pra rede neural que analisam as características pra fazer a previsão na camada de saída.



Esse é basicamente o passo a passo da rede convolucional, a imagem entra e passa por várias camadas de convolução e pooling pra depois a matriz que saiu disso tudo vira um vetor e é jogada pra rede neural densa que vai tomar as decisões e fazer a previsão.

Quanto mais dados uma rede analisa melhor ela fica, porém de vez em quando podemos ter falta de imagens pra treinar o modelo, pra isso nos pegamos

as imagens que já temos e giramos ela em diferentes formas, pois pra rede quando mudamos algo de lugar ainda fica difícil pra ela prever corretamente, isso é chamado de Data augmentation.

Antes de enviarmos dados pra rede é feito a normalização deles, porém com a medida que os dados vão passando de camada em camada essa normalização vai se perdendo, pra isso foi criado o BatchNormalization que fica entre as camadas de convolução, ela pega os grupos de dados e calcula a média e o desvio padrão e aplica aos dados pra normalização, deixando o treinamento mais estável e reduz o overfitting.

Anteriormente foi ensinado que pra uma rede neural conseguir identificar dados categóricos nós precisamos usar o One Hot Encoding pra transformar categorias em uma matriz com 0 e 1 pra rede entender, porém pra interpretar palavras fica mais complicado usar essa técnica pois precisaríamos de uma matriz gigante no número exato de palavras daquela língua que estamos tentando interpretar. A técnica de embedding ajuda nisso, ela faz com que cada palavra seja representada por um conjunto de números, cada palavra recebe um número inteiro como se fosse um ID pra quando a rede precisar pegar esse número e ir na matriz de embedding pra checar o conjunto de números que representa aquela palavra, essa técnica é muito usada pois ela mantém a proximidade de palavras parecidas, por exemplo, rei e rainha ficam próximos em questão de números pois são parecidas, mas ficam longe da palavra carro.

1	2	3	4	5	6	7	8
The	beatles	e	a	melhor	banda	do	mundo
1	0,23	0,25		0,30	0,32		
2	0,57	0,60		0,67	0,70		
3	0,2	0,1		0,4	0,6		
4	0,10	0,12		0,16	0,19		
5	0,40	0,44		0,48	0,50		
6	...	...		...	...		
7	...	...		...	...		
8	...	...		...	...		

Depois ele aborda rapidamente sobre as arquiteturas de redes convolucionais, onde ele fala que toda rede tem as camadas convolucionais,

primeiro pra processar a imagem e depois vem as camadas densas transformando os dados em um vetor, basicamente o fluxo que um insight visual visto anteriormente mostra. Ele também fala sobre arquiteturas famosas e como elas seguem o padrão que ele ensina nas aulas:

- VGG: O vgg tem várias formas mas que alteram somente o número de camadas, no exemplo do vídeo ela tem 16 camadas ao total, sendo 13 só pra convolução, ele divide 2 camadas de convolução air maxpool depois do maxpool ela vai pra mais 2 de convolução depois pro maxpool de novo, e assim vai indo até chegar na camada densa pra realizar a saída.
- AlexNet: Ele é menor que o vgg pois foi criado dois anos antes, ele tem só 8 camadas no total sendo 5 de convolução e 3 densas
- GoogleLeNet: Tem 22 camadas e traz o módulo inception que é basicamente ao invés de decidir um tamanho de filtro ele passa vários filtros de vários tamanhos diferentes ao mesmo tempo e junta os resultados e a própria rede que vai decidir as características mais importantes.

O vídeo apresenta o Mean Squared Error que é uma forma de calcular o erro das previsões, onde o erro é elevado ao quadrado pra sempre manter ele positivo, pois ele funciona com 0 sendo o indicador que o modelo não errou nada então quanto mais perto de 0 o erro estiver melhor foram as previsões.

Se o modelo faz previsões binárias uma forma pra calcular o erro dela é usando Binary Cross Entropy, dando um exemplo, a rede quer prever se a imagem é de uma pessoa ou não, e numa previsão ela fala que tem 0.95% de chance de ser uma pessoa e na imagem era realmente uma pessoa ela acertou e com confiança na resposta, porém se ela fala que tem 0.05% de ser uma pessoa e na foto era uma pessoa ela sofre uma penalização gigante. Esse método consegue fazer isso pois ele usa a distribuição de bernoulli diferente da anterior que usa a distribuição normal.

Pra problemas com mais de uma classe é usado o Categorical Cross Entropy, onde ele dá uma probabilidade pra cada classe que existe na rede, supondo que a rede quer prever se na foto tem um gato, cachorro e zebra, e a imagem correta tem um gato esse método olha somente pra previsão da rede pra classe correta da imagem e a penaliza de acordo.

A descida do gradiente é uma forma de ajustar os pesos de uma rede, primeiro ela calcula o erro e depois a derivada que é o gradiente do erro em relação ao peso do neurônio o resultado desse cálculo mostra pra onde o erro aumenta então a rede coloca os pesos na direção oposta pra diminuir o erro, e isso é repetido várias vezes durante o treinamento.

O Stochastic Gradient Descent segue a mesma lógica do método anterior soque ele pega uma amostra dos dados e faz o cálculo da média do grupo pois o resultado vai ser próximo da média real, ele separa em vários grupinhos e vai passando de grupo em grupo e a cada passada ele faz os cálculos e atualiza os pesos.

Depois ele fala sobre o Momentum que é uma técnica que aprimora a descida do gradiente, ela basicamente pega um pouco do ajuste anterior e usa no ajuste novo pra acelerar o processo. Existem técnicas que melhorar ainda mais rápido a taxa de aprendizado que consiste em iniciar com o valor da taxa muito alto e descer ele rapidamente e quando chegar perto de 0 desacelerar existem técnicas prontas que fazem isso de forma automática como o Adagrad onde ele guarda os valores do gradiente da rede e se ele tem gradientes muito altos ele diminui a taxa e se o gradiente é pequeno e quase não muda ele coloca uma taxa maior, o problema é que essa tecnica faz com que a taxa de aprendiza chegue perto de 0 muito rapido fazendo que o modelo para de aprender quando ele ainda poderia melhorar. O RMSprop guarda os valores do gradiente igual o adagrad mas faz pegando pegando parte do cache antigo com parte do gradiente novo fazendo com que os gradientes muito antigos vao sendo esquecidos.

O Adam junta o momentum e o RMSdrop, usando uma média móvel pra ver pra onde o gradiente ta indo ele deixa duas médias guardadas onde a primeira guarda a dos gradientes e a segunda guarda a dos gradientes ao quadrado, fazendo assim o aprendizado ser mais rápido.

Na minha parte prática resolvi usar um dataset do tensorflow chamado CIFAR100 que tem 100 categorias e 60 mil imagens coloridas pra rede prever oque fica difícil pra ela, eu comecei normalizando os pixel dividindo por 255 e comecei a fazer a rede, coloquei 2 camadas de convolução uma com 64 filtros e outra com 128 as duas usam dropout 20% pra uma e 30% pra outra, maxpooling nas duas e batchnormalization, depois da convolução ela passa os dados pra uma rede neural densa com 512 neurônios com uma camada de saída com 100 porque sao 100 categorias.

Eu rodei a rede 2 vezes pra tentar melhorar a acurácia e o erro e fiz alguns gráficos pra mostrar ambos.

3. Conclusão

Em resumo o card ensina todos os conceitos muito bem, ele ajuda a entender melhor redes neurais convolucionais e todas as técnicas em volta disso, gostei da parte sobre o embedding que é uma técnica útil pra textos. A parte prática do card é muito boa mostrando os conceitos e os resultados aplicados realmente.